SMART CONTRACT AUDIT REPORT

for

cuniBTC

Prepared By: Xiaomi Huang

PeckShield

May 6, 2026

Document Properties

Client	BedRock
Title	Smart Contract Audit Report
Target	cuniBTC
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	May 6, 2026	Xuxian Jiang	Final Release
1.0-rc	May 5, 2026	Xuxian Jiang	Release Candidate

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1 Introduction 4
- 1.1 About cuniBTC . . . 4
- 1.2 About PeckShield . . . 5
- 1.3 Methodology . . . 5
- 1.4 Disclaimer . . . 7

2 Findings 9
- 2.1 Summary . . . 9
- 2.2 Key Findings . . . 10

3 Detailed Results 11
- 3.1 Improved Redeem Delay Use in DelayRedeemRouter . . . 11
- 3.2 Optimized _getDebtTokenAmount() Logic in DelayRedeemRouter . . . 12
- 3.3 Possible Arithmetic Overflow Avoidance in Airdrop And Vault . . . 15
- 3.4 Accommodation of Non-ERC20-Compliant Tokens . . . 16
- 3.5 Trust Issue Of Admin Keys . . . 17

4 Conclusion 20

References 21

1 Introduction

Given the opportunity to review the design document and related smart contract source code of the cuniBTC protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About cuniBTC

Bedrock Vault is a permissionless, smart contract-based system designed to improve Bitcoin capital efficiency via uniBTC. It aggregates user deposits, deploys them into external yield strategies, and distributes returns proportionally within an epoch-based framework. Strategy execution may be supported by AI-assisted systems, while all fund flows are governed on-chain. The basic information of audited contracts is as follows:

Table 1.1: Basic Information of Audited Contracts

Item	Description
Target	cuniBTC
Website	https://bedrock.technology/
Type	EVM Smart Contract
Language	Solidity
Audit Method	Whitebox
Latest Audit Report	May 6, 2026

In the following, we show the Git repository of reviewed files and the commit hash values used in this audit.

- https://github.com/Bedrock-Technology/uniBTC.git (9eb2c1e)

And this is the commit ID after all fixes for the issues found in the audit have been checked in:

- https://github.com/Bedrock-Technology/uniBTC.git (1e66163)

1.2 About PeckShield

PeckShield Inc. [9] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification

Impact \ Likelihood	*High*	*Medium*	*Low*
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [8]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [7], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the implementation of the `cuniBTC` protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logic, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	2	■■
Low	2	■■
Informational	1	■
Total	5	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in Section 3.

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 2 medium-severity vulnerabilities, 2 low-severity vulnerabilities, and 1 informational recommendation.

Table 2.1: Key cuniBTC Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Medium	Improved Redeem Delay Use in DelayRedeemRouter	Business Logic	Resolved
PVE-002	Informational	Optimized _getDebtTokenAmount() Logic in DelayRedeemRouter	Coding Practices	Resolved
PVE-003	Low	Possible Arithmetic Overflow Avoidance in Airdrop And Vault	Numeric Issues	Resolved
PVE-004	Low	Accommodation of Non-ERC20-Compliant Tokens	Business Logic	Resolved
PVE-005	Medium	Trust Issue Of Admin Keys	Security Features	Mitigated

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to Section 3 for details.

3 Detailed Results

3.1 Improved Redeem Delay Use in DelayRedeemRouter

- ID: PVE-001
- Severity: Medium
- Likelihood: Medium
- Impact: Medium
- Target: `DelayRedeemRouter`
- Category: Business Logic [6]
- CWE subcategory: CWE-841 [3]

Description

The `cuniBTC` protocol has a core `DelayRedeemRouter` contract that allows users to redeem for specified `BTC` tokens. The redemption is subject to a `redeemDelay` period (measured in timestamps) before the redemption can be claimed. In the process of examining the redemption logic, we notice it makes use of latest `redeemDelay` period, instead of the `redeemDelay` period at the time when the redemption is requested.

To elaborate, we show below the implementation of the related `_claimDelayedRedeems()` routine. As the name indicates, it claims the delayed user redeems after the `redeemDelay` period is expired. However, it comes to our attention that the `redeemDelay` period being used for claimable validation is the one currently in effect (line 810), not the one when the redemption request is made. As a result, a redeeming user may be blocked if a large `redeemDelay` period is applied after the redemption request is made.

```
802     function _claimDelayedRedeems(address recipient, uint256
            maxNumberOfDelayedRedeemsToClaim) internal {
803         uint256 delayedRedeemsCompletedBefore = _userRedeems[recipient].
                delayedRedeemsCompleted;
804         uint256 numToClaim = 0;
805         DebtTokenAmount[] memory debtAmounts;
806
807         //==========================================================================
808         // 1. Get the length of debt that need to be repaid and the amount of each type
                of debt.
809         //==========================================================================
```

```
        (numToClaim, debtAmounts) = _getDebtTokenAmount(
            recipient, delayedRedeemsCompletedBefore, redeemDelay,
                maxNumberOfDelayedRedeemsToClaim
        );
        ...
    }
```

Listing 3.1: `DelayRedeemRouter::_claimDelayedRedeems()`

Recommendation Revisit the above logic to make use of the intended `redeemDelay` period when finalizing user redemption requests.

Status This issue has been resolved as the team confirms it is part of protocol design.

3.2 Optimized _getDebtTokenAmount() Logic in DelayRedeemRouter

- ID: PVE-002
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: `DelayRedeemRouter`
- Category: Business Logic [6]
- CWE subcategory: CWE-841 [3]

Description

As mentioned earlier, the `cuniBTC` protocol has a built-in `DelayRedeemRouter` contract to handle user redemption requests. Accordingly, it provides a key helper routine to get the claimable debt list. In the process of examining the helper routine to calculate the claimable debt list, we notice current implementation can be improved.

To elaborate, we show below the implementation of the related `_getDebtTokenAmount()` routine. As the name indicates, this routine is used to obtain the claimable debt list. It comes to our attention that if the number of unique token types in the claimable debt list matches the total number of delayed redemption requests, the routine directly returns `(numToClaim, debtAmounts)` (line 977). When the delayed redemption requests involve the same token type, the returned list is shrink-ed by making the return list with distinct token types. And the shrink operation can be simply achieved with `assembly {mstore(debtAmounts, tokenCount)}`, instead of current `for`-loop (lines 981-984).

```
917     function _getDebtTokenAmount(
918         address recipient,
919         uint256 delayedRedeemsCompletedBefore,
920         uint256 delayTimestamp,
921         uint256 maxNumberOfDelayedRedeemsToClaim
922     ) internal view returns (uint256, DebtTokenAmount[] memory) {
```

```
        uint256 redeemsLength = _userRedeems[recipient].delayedRedeems.length;
        uint256 numToClaim = 0;

        // ==========================================================================
        // 1. Check how many debts can be redeemed.
        // ==========================================================================
        while (
            numToClaim < maxNumberOfDelayedRedeemsToClaim
                && (delayedRedeemsCompletedBefore + numToClaim) < redeemsLength
        ) {
            // Copy the delayedRedeem from storage to memory.
            DelayedRedeem memory delayedRedeem =
                _userRedeems[recipient].delayedRedeems[delayedRedeemsCompletedBefore +
                    numToClaim];

            // Check if each delayedRedeem is claimable, and exit the loop immediately
            // once a non-claimable delayedRedeem is encountered.
            if (block.timestamp < delayedRedeem.createdAt + delayTimestamp) {
                break;
            }

            // Increment i to reflect the processing of the current delayedRedeem being
                claimed.
            unchecked {
                ++numToClaim;
            }
        }

        // ==========================================================================
        // 2. Count the types of debt and the amount of each debt type.
        // ==========================================================================
        if (numToClaim > 0) {
            DebtTokenAmount[] memory debtAmounts = new DebtTokenAmount[](numToClaim);
            uint256 tokenCount = 0;
            for (uint256 i = 0; i < numToClaim; i++) {
                DelayedRedeem memory delayedRedeem =
                    _userRedeems[recipient].delayedRedeems[delayedRedeemsCompletedBefore
                        + i];
                bool found = false;
                uint256 fee = _getFee(recipient, delayedRedeemsCompletedBefore + i);
                for (uint256 j = 0; j < tokenCount; j++) {
                    if (debtAmounts[j].token == delayedRedeem.token) {
                        debtAmounts[j].amount += delayedRedeem.amount;
                        debtAmounts[j].fee += fee;
```

```
                        found = true;
                        break;
                    }
                }
                if (!found) {
                    debtAmounts[tokenCount] =
                        DebtTokenAmount({token: delayedRedeem.token, amount:
                            delayedRedeem.amount, fee: fee});
                    tokenCount++;
                }
            }

            // The number of unique token types matches the total number of delayed
                redemption requests.
            if (tokenCount == debtAmounts.length) {
                return (numToClaim, debtAmounts);
            }

            // Some delayed redemption requests involve the same token type.
            DebtTokenAmount[] memory finalAmounts = new DebtTokenAmount[](tokenCount);
            for (uint256 k = 0; k < tokenCount; k++) {
                finalAmounts[k] = debtAmounts[k];
            }
            return (numToClaim, finalAmounts);
        }

        return (0, new DebtTokenAmount[](0));
    }
```

Listing 3.2: `DelayRedeemRouter::_getDebtTokenAmount()`

Recommendation Simplify the above-mentioned routine to properly remove the redundant entries, if any.

Status This issue has been resolved as the team confirms only one token type is currently supported.

3.3 Possible Arithmetic Overflow Avoidance in Airdrop And Vault

- ID: PVE-003
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `Airdrop`, `Vault`
- Category: Coding Practices [5]
- CWE subcategory: CWE-1126 [1]

Description

DeFi protocols typically have a number of system-wide parameters that can be dynamically configured on demand. The `cuniBTC` protocol is no exception. Specifically, if we examine the `Vault` contract, it has defined a number of protocol-wide risk parameters, such as `operatePeriod` and `lockupPeriod` for each specific token. In the following, we show the corresponding setter routine that dynamically configures these risk parameters.

```
    function setPeriod(uint256 _start, uint256 _operatePeriod, uint256 _lockupPeriod)
        external onlyRole(OPERATOR_ROLE) {
        require(_operatePeriod + _lockupPeriod > 0, "USR018");
        require(_start <= block.number, "USR019");
        startGenesis = _start;
        operatePeriod = _operatePeriod;
        lockupPeriod = _lockupPeriod;
        emit PeriodSet(_start, _operatePeriod, _lockupPeriod);
    }
```

Listing 3.3: `Vault::setPeriod()`

The above setter can be improved to explicitly ensure the given parameters do not result in unexpected arithmetic overflows. In particular, the `operatePeriod` and `lockupPeriod` will be later added to determine and alternate two different periods, i.e., `OperatePeriod` and `LockupPeriod`. With that, we need to ensure their sum does not result in an arithmetic overflow.

Recommendation Improve the above routine to ensure the given risk parameters fall in the intended ranges. Note there are two other functions, i.e., `submitRoot()` and `updateDuration()` in the `Airdrop` contract can be similarly improved.

Status This issue has been fixed in the following commit: `95c289a`.

3.4 Accommodation of Non-ERC20-Compliant Tokens

- ID: PVE-004
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `AirdropDistributor`
- Category: Business Logic [6]
- CWE subcategory: N/A

Description

Though there is a standardized ERC-20 specification, many token contracts may not strictly follow the specification or have additional functionalities beyond the specification. In this section, we examine the `transfer()` routine and possible idiosyncrasies from current widely-used token contracts.

In particular, we use the popular stablecoin, i.e., `USDT`, as our example. We show the related code snippet below. Specifically, the `transfer()` routine does not have a return value defined and implemented. However, the `IERC20` interface has defined the `transfer()` interface with a `bool` return value. As a result, the call to `transfer()` may expect a return value. With the lack of return value of `USDT`'s `transfer()`, the call will be unfortunately reverted.

```
    function transfer(address _to, uint _value) public onlyPayloadSize(2 * 32) {
        uint fee = (_value.mul(basisPointsRate)).div(10000);
        if (fee > maximumFee) {
            fee = maximumFee;
        }
        uint sendAmount = _value.sub(fee);
        balances[msg.sender] = balances[msg.sender].sub(_value);
        balances[_to] = balances[_to].add(sendAmount);
        if (fee > 0) {
            balances[owner] = balances[owner].add(fee);
            Transfer(msg.sender, owner, fee);
        }
        Transfer(msg.sender, _to, sendAmount);
    }
```

Listing 3.4: USDT::**transfer**()

Because of that, a normal call to `transfer()` is suggested to use the safe version, i.e., `safeTransfer()`, In essence, it is a wrapper around ERC20 operations that may either throw on failure or return false without reverts. Moreover, the safe version also supports tokens that return no value (and instead revert or throw on failure). Note that non-reverting calls are assumed to be successful.

In current implementation, if we examine the `cuniBTC::withdraw()` routine that is designed to withdraw tokens that may be accidently sent to the `cuniBTC` token contract. To accommodate the specific idiosyncrasy, there is a need to user `safeTransfer()`, instead of `transfer()` (line 69).

```
    function withdraw(address _token, address _to, uint256 _amount) external onlyRole(
        DEFAULT_ADMIN_ROLE) {
        if (_token == address(0)) {
            payable(_to).transfer(_amount);
        } else {
            IERC20(_token).transfer(_to, _amount);
        }
    }
```

Listing 3.5: cuniBTC::withdraw()

Recommendation Accommodate the above-mentioned idiosyncrasy about ERC20-related `approve()/transfer()/transferFrom()`.

Status This issue has been fixed in the following commit: `9224ac4`.

3.5 Trust Issue Of Admin Keys

- ID: PVE-005
- Severity: Medium
- Likelihood: Medium
- Impact: Medium
- Target: `Multiple Contracts`
- Category: Security Features [4]
- CWE subcategory: CWE-287 [2]

Description

In the `cuniBTC` protocol contract, there is a privileged account (with the assigned `DEFAULT_ADMIN_ROLE` role) that plays a critical role in governing and regulating the DAO-wide operations (e.g., assign roles, configure parameters, and upgrade proxies). In the following, we show the representative functions potentially affected by the privilege of this account.

```
    function execute(address _target, bytes memory _data, uint256 _value)
        external
        nonReentrant
        onlyRole(OPERATOR_ROLE)
        serviceNormal
        returns (bytes memory)
    {
        require(allowedTargetList[_target], "SYS001");
        return _target.functionCallWithValue(_data, _value);
    }
    ...
    function allowToken(address[] memory _token) external onlyRole(DEFAULT_ADMIN_ROLE) {
        for (uint256 i = 0; i < _token.length; i++) {
            allowedTokenList[_token[i]] = true;
        }
        emit TokenAllowed(_token);
```

```
    }
    ...
    function denyToken(address[] memory _token) external onlyRole(DEFAULT_ADMIN_ROLE) {
        for (uint256 i = 0; i < _token.length; i++) {
            allowedTokenList[_token[i]] = false;
        }
        emit TokenDenied(_token);
    }
    ...
    function allowTarget(address[] memory _targets) external onlyRole(DEFAULT_ADMIN_ROLE
        ) {
        for (uint256 i = 0; i < _targets.length; i++) {
            allowedTargetList[_targets[i]] = true;
        }
        emit TargetAllowed(_targets);
    }
    ...
    function denyTarget(address[] memory _targets) external onlyRole(DEFAULT_ADMIN_ROLE)
         {
        for (uint256 i = 0; i < _targets.length; i++) {
            allowedTargetList[_targets[i]] = false;
        }
        emit TargetDenied(_targets);
    }
    ...
    function setCap(address _token, uint256 _cap) external onlyRole(DEFAULT_ADMIN_ROLE)
        {
        require(_token != address(0x0), "SYS003");
        uint8 decs = ERC20(_token).decimals();
        require(decs == 8  decs == 18, "SYS004");
        tokenCaps[_token] = _cap;
        emit CapSet(_token, _cap);
    }
    ...
    function setTotalSupply(uint256 _totalSupply) external onlyRole(DEFAULT_ADMIN_ROLE)
        {
        totalSupply = _totalSupply;
        emit TotalSupplySet(totalSupply);
    }
```

Listing 3.6: Example Privileged Operations in `Vault`

We emphasize that the privilege assignment may be necessary and consistent with the protocol design. However, it is worrisome if the privileged account is not governed by a DAO-like structure. Note that a compromised account would allow the attacker to modify a number of sensitive vault parameters, which directly undermines the assumption of the vault design.

In the meantime, the vault contract and others make use of the proxy contract to allow for future upgrades. Their upgrade is a privileged operation, which also falls in this trust issue on the admin key.

Recommendation Promptly transfer the privileged account to the intended DAO-like governance contract. All changed to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

Status The issue has been confirmed and will be mitigated with the use of a multi-sig to manage the privileged account.

PeckShield

4 Conclusion

In this audit, we have analyzed the design and implementation of the `cuniBTC` protocol, which is designed to improve `Bitcoin` capital efficiency. In particular, it aggregates user deposits, deploys them into external yield strategies, and distributes returns proportionally within an epoch-based framework. Strategy execution may be supported by `AI`-assisted systems, while all fund flows are governed on-chain. The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and addressed.

Meanwhile, we need to emphasize that smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

PeckShield

References

[1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. https://cwe.mitre.org/data/definitions/1126.html.

[2] MITRE. CWE-287: Improper Authentication. https://cwe.mitre.org/data/definitions/287.html.

[3] MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. https://cwe.mitre.org/data/definitions/841.html.

[4] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[5] MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

[6] MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

[7] MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

[8] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[9] PeckShield. PeckShield Inc. https://www.peckshield.com.